roife

[redacted] | roifewu@gmail.com | roife | roife.github.io

Education

Nanjing University
2023.09 - 2026.06 (expected)

Master's Degree in *Computer Science and Technology* | PASCAL Lab. Tutor: [redacted] Li | Focus on **PL** and **Program Analysis**.
TA: **Principles and Techniques of Compilers** (Spring 2024)

Beihang University
2019.09 - 2023.06

Bachelor's Degree in *Computer Science and Technology* | GPA 3.84/4.00, qualified for postgraduate recommendation.
TA: **Programming in Practice** (Fall 2020), **Object-oriented Design and Construction** (Fall 2021, Spring 2022 | S.T.A.R. team)

Work Experience

NVIDIA
2025.02 - Present

OCG (Optimizing Code Generator) team — GPU Compiler LLVM Backend Intern

- Participating in unifying the load-store instruction vectorizer between NVIDIA GPU graphics compiler and NVVM, ensuring the graphics compiler's vectorizer aligns with LLVM upstream:
  - Designed an encoding scheme for **multi-address graphic memory instructions** for LLVM's memory-access instructions vectorizer, implementing vectorization for multiple GPU graphic memory instructions while minimizing divergence from upstream;
  - Added several GPU memory instruction vectorization optimizations, including support for irregular memory access sequence vectorization and integer address vectorization; participated in implementing a new memory instruction alignment width inference pass, improving vectorizer performance while reducing codebase coupling;

Rust Foundation Fellowship Program
2024.09 - 2025.09

Rust Foundation Fellowship (about 20 people globally) — Project Fellow

- As a member of the **Rust-lang Organization** (rust-analyzer-contributors-team) and one of the rust-lang/**rust-analyzer** (official Rust IDE) **maintainers**, ranked in the **top 1%** of contributors; participated in issues handling and PR reviews across most project modules; also contributed to other projects in the Rust language community, such as rust-lang/**rust-clippy**
- Implemented features like control flow highlighting, snapshot test updates, and participated in numerous bug fixes, enhancing IDE capabilities in code understanding and auto-generation;
- Wrote a **SIMD** implementation for the unicode line breaking module for ARM NEON, achieving a **6.5x** speedup;
- **Emergency incident response for v0.3.1992**: 4 hours after release, the community discovered a critical bug causing resource exhaustion. I identified the issue **in 3 hours** and designed a new algorithm as fix. This emergency fix controlled the incident's impact.

Awards

- 2022 **National Scholarship** (ranked 1/195 in the major), **Outstanding Graduate** of Beihang University;
- **First Prize** in the 2021 NSCSCC Compilation System Design Competition (Huawei Bisheng Cup), ranking 2nd overall;
- **First Prize** in the Lanqiao Cup C++ Programming Contest (Beijing Division) and **Third Prize** in the National Finals;
- Additionally awarded over ten provincial and university-level awards and scholarships.

Projects

Vizsla
roife/**vizsla** (WIP)

Modern IDE for Chip Frontend Design · Master's Thesis Project — Rust / SystemVerilog

- Implemented a **semantic analysis framework** and IDE infra for SV, aiming to equip chip design with modern IDE features;
- Based on an **incremental computation** architecture, designed and implemented an incremental analysis IR and specialized passes for efficient and accurate on-demand analysis;
- Project achieves **industry-leading standards** in functionality, performance, and usability: completed **dozens of** modern IDE features for SystemVerilog including code navigation, semantic refactoring, and completion with **millisecond-level** latency;
- Based on the Language Server Protocol, ensuring compatibility with VS Code, Emacs, NeoVim, and other mainstream editors.

Ailurus
roife/**ailurus** (WIP)

Experimental Programming Language and Toolchain Design · Personal Interest Project — Rust

- Based on **Martin-Löf type theory**, supporting **dependent types**, dependent pattern matching, and inductive datatypes. Implements propositional equality and uses Normalization by Evaluation for equivalence checking, enabling simple theorem proving;
- Features **typeclass-based ad-hoc polymorphism** with **operator overloading** for flexible code reuse;
- Implemented a **module system** for namespace management and encapsulation, addressing code organization in large projects;

- Serves as an experimental platform to explore collaborative design architectures for modern programming language toolchains (compilers, IDEs), aiming to enhance development efficiency and maintainability.

Ayame

No-SF-Work/**ayame**

Compiler from SysY (C subset) to ARMv7 · Bisheng Cup Competition Project — Java / LLVM-IR / ARM

- Collaborative project, primarily responsible for backend optimizations on Machine IR, including **iterative register coalescing**, **instruction scheduling**, dead code elimination, and peephole optimizations. Also contributed to syntax tree visitor;
- Handled project testing and DevOps, setting up workflows with Docker and GitLab CI, and writing Python for automated testing;
- The project, built from scratch, featured a complete compiler pipeline (parsing to code generation) with extensive SSA IR and Machine IR optimizations. It ranked **2nd overall** in the competition, achieving **1st place in nearly half of the testcases** and outperforming `gcc -O3` and `clang -O3` on 1/3 of the examples.

LLVM-Lite

roife/**llvm-lite**

Lightweight Edge-side Compiler for Neural Network Operators · Undergraduate Thesis Project — C++ / LLVM-IR

- Aimed to utilize known **shape information** from edge inference devices for **secondary optimization** of deep learning operators, reducing runtime spatial and temporal overhead;
- Included a **lightweight compiler** on inference devices and **trimming** work of the LLVM Codegen module. For target workloads, implemented optimizations such as **SCCP** and **DCE**, minimize overhead while maximizing results;
- Received **excellent** evaluation for the thesis. Successfully reduced inference time by 6% and binary file size by 38% for convolution and softmax operators; implemented **parse-time optimizations** that reduced compilation time by 60% and memory usage by 60%.

Other Personal Projects

- Caniformia/**HangGai** (Vue/RoR / SwiftUI, collaborative) Learning app for courses in BUAA, available on the App Store
- roife/**firefly** (Rust) A simple neural network training/inference framework, implementing convolution, fully connected layers and other operators, with MNIST dataset classification implemented;
- roife/**mole** (Verilog / MIPS) A five-stage pipelined CPU, implementing **50+** instructions with **forwarding** and **stalling** mechanisms; also implemented coprocessor CP0 to handle **interrupts** and **exceptions**;
- roife/**mos** (C / MIPS) An OS kernel with an **exokernel** design, implementing modules for memory mapping, process management, file system, system calls, and a shell;

Open Source Contributions

- **Rust Organization** (rust-analyzer contributors team) member, primarily maintaining rust-lang/**rust-analyzer** also contributed to rust-lang/**rust**, rust-lang/**rust-clippy**, rust-lang/**rustup**, rust-lang/**rust-mode**
- llvm/**llvm-project**, clangd/**vscode-clangd**, MikePopoloski/**slang**, google/**autocxx**, yuin/**goldmark**, moonbitlang/**tree-sitter-moonbit**, more projects on GitHub.

Skills

- **Programming Languages**: Not tied to any specific language. Especially proficient in C, C++, Rust, Java, Python, JavaScript/TypeScript, Verilog/SystemVerilog, and EmacsLisp; have also worked with Ruby, Swift, OCaml, Haskell, Coq, Agda, etc.
- **Programming Language Theory**: Familiar with type theory, formal semantics, and the theory of computation; experienced with **interactive theorem provers**. Knowledge of the theory and implementation of **type systems** (e.g., Hindley-Milner, Dependent Types).
- **Static Analysis**: Familiar with **static analysis** algorithms (e.g., CFA, IFDS, pointer analysis with varying sensitivities).
- **Compiler Design**: **3 YoE**, proficient in the full compiler pipeline, with an emphasis on **compiler optimizations**:
  - Experience implementing PL features across multiple paradigms, such as bidirectional type checking and module systems.
  - Familiar with various **IRs** (e.g., SSA, MLIR, etc.) and **optimizations** across stages (e.g., Mem2Reg, SCEV, register allocation, etc.).
  - In-depth knowledge of **LLVM** and LLVM-IR; have implemented several analysis and optimization passes.
- **IDE Development**: **2 YoE**. Familiar with IDE architectures based on **incremental computation** (esp. rust-analyzer and clangd); versed in the LSP and plugin development for VS Code, Emacs, and other editors.
- **Computer Architecture**: Familiar with ARM, x86, etc.; understanding of OoO execution, etc.; knowledgeable about NVIDIA GPU.
- **Application Development**: Proficient with web backend frameworks (Ruby on Rails, Django), frontend (Vue.js), and iOS (SwiftUI) development. Experienced with database design (PostgreSQL, Redis) and DevOps (Docker, CI/CD).
- **Development Environment**: Proficient in Emacs and VS Code; comfortable with macOS and Linux; adept at leveraging GenAI.

Misc

- **Club**: Served as President of *the Beihang OpenAtom Open Source Club*, organizing multiple technical sharing and exchange events;
- **Blog**: roife.github.io mainly focused on theoretical computer science;
- **Languages**: Chinese (native), English.